Asymptotic Efficiency

$\log_a n \in \Theta(\log n)$ $a>1$

$a_k n^k + a_{k-1} n^{k-1} + \dots + a_0 \in \Theta(n^k)$

$\log n < n^2 < a^n < n! < n^n$

Summation Formulas

$\sum_{i=e}^{u} 1 = u - e + 1 \quad (e \le u)$

$\sum_{i=1}^{n} 1 = n$

$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{1}{3} n^3$

$\sum_{i=1}^{n} i^k \approx \frac{1}{k+1} \cdot n^{k+1}$

$\sum_{i=0}^{n} a^i = \frac{a^{n+1}-1}{a-1} \quad (a \neq 1)$

$\sum_{i=0}^{n} 2^i = 2^{n+1} - 1$

$\sum_{i=1}^{n} i \cdot 2^i = (n-1)2^{n+1} + 2$

$\sum_{i=1}^{n} \frac{1}{i} \approx \ln n + \gamma \quad (\gamma \approx 0.5772)$

$\sum_{i=1}^{n} \log i \approx n \cdot \log n$

Sum Manipulation

$\sum_{i=e}^{u} c a_i = c \sum_{i=e}^{u} a_i$

$\sum_{i=e}^{u} (a_i \pm b_i) = \sum_{i=e}^{u} a_i \pm \sum_{i=e}^{u} b_i$

$\sum_{i=e}^{u} a_i = \sum_{i=e}^{m} a_i + \sum_{i=m+1}^{u} a_i \quad e \le m < u$

$\sum_{i=e}^{u} (a_i - a_{i-1}) = a_u - a_{e-1}$

Floor and Ceiling

$x-1 < \lfloor x \rfloor \le x \le \lceil x \rceil < x+1$

$\lfloor x+n \rfloor = \lfloor x \rfloor + n \quad \lceil x+n \rceil = \lceil x \rceil + n$
(for real x, integer n)

$\lfloor n/2 \rfloor + \lceil n/2 \rceil = n$

$\lceil \log(n+1) \rceil = \lfloor \log n \rfloor + 1$

$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$ as $n \to \infty$

$(n+m) \bmod p = (n \bmod p + m \bmod p) \bmod p$

Master Theorem

$T(n) = aT(\frac{n}{b}) + f(n)$

$f(n) \in \Theta(n^d), \ d \ge 0$

$a < b^d : T(n) \in \Theta(n^d)$

$a = b^d : T(n) \in \Theta(n^d \log n)$

$a > b^d : T(n) \in \Theta(n^{\log_b a})$

Results are the same for O

Combinatorics

$P(n) = n!$

$C(n,k) = \frac{n!}{k!(n-k)!}$

of subsets 2^n

Linear Homegeneos Recurrence

$aX(n) + bX(n-1) + cX(n-2) = 0$

CE:

$ar^2 + br + c = 0$

$r = \frac{-b \pm \sqrt{D}}{2a}$, 2 distinct roots

$X(n) = \alpha r_1^n + \beta r_2^n$

$X(n) = ar^n + \beta n r^n$

Fibonacci

$F(n) = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2}\right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2}\right)^n$

$\begin{bmatrix} F(n-1) & F(n) \\ F(n) & F(n+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}^n \quad n \ge 1$

Analysis of Algorithms

Sebction sort $\in \Theta(n^2)$

BFSeqSearch: $C_B(n,m) = m$

$C_W(n,m) = m(n-m+1)$

BFPolEv $\in \Theta(n^2)$ R⇉L:

$\in \Theta(n)$ $a_n x^n + \dots + a_1 x^1$

BFClosestPair $\in \Theta(n^2)$

DFS: stack adj mat.:

$\in \Theta(V^2)$ adj list: $\in \Theta(|V|+|E|)$

BFS: queue adj mat: $\in \Theta(V^2)$

adj list: $\in \Theta(|V|+|E|)$

Insertion sort: $C_W \in \Theta(n^2)$ stabil

Binary search: $C_W \in \lceil \log_2(n+1) \rceil$

$C_W^{alt} = 1 + C_W(\lfloor n/2 \rfloor)$

Quick select: Avg: $C(n) = C(\frac{n}{2}) + (n+1)$

$\in \Theta(n)$ Worst: $C(n) = (n-1) + (n+1)$

$\in \Theta(n^2)$

Mergesort $\in O(n \cdot \log n)$ $\Theta(n \cdot \log n)$

$\Theta(n)$ place

$\log(2n!) \in \Theta(n \cdot \log n)$

Decrease and Conquer

Dec. by const: insertion sort, topological sort, algos for gen. perms. and subsets (dec by 1 usually)

Dec by const factor (usually by half): binary search and bisection method, exponentiation by squaring, multiplication a la russe

Variable size decrease: Euclid's algo., selection by partit., Nim-like games

Brute force: $a \cdot a \cdot a \cdot a \dots a$

Divide&Conq: $a^{\lfloor \frac{n}{2} \rfloor} \cdot a^{\lceil \frac{n}{2} \rceil}$

Decrease by 1: $a^{n-1} \cdot a$ if even: $(a^{\frac{n}{2}})^2$

Decrease by constant factor: odd: $(a^{\lfloor \frac{n}{2} \rfloor})^2 \cdot a$

DFS based topological sort: Perform DFS, noting order of vertices popped of the stack, reverse order.

back edges X not dag dag: directed acyclic graph

Generate perms: 1 | 12 21 | 321 231 213 (dec by 1)

000 001 011 010 ⇐ 00 01 } 00 01 11 10

110 111 101 100 ⇐ 11 10 }

Binarysearch: sorted array

Russianpeasant multip.: $n \cdot m = \frac{n}{2} \cdot 2m$ (even n)

$n \cdot m = \frac{n-1}{2} \cdot 2m + m$ if $n>1$ and mif $n \ge 1$ (odd n)

$\gcd(m,n) = \gcd(n, m \bmod n)$ $T(n) \in O(\log n)$

Interpolation search: sorted array

$x = l + \lfloor (u - A[l])(r-l) / A[r] - A[l] \rfloor$

BST: $C_w(n) = n$ $C_{avg}(n) \approx 2 \ln n \approx 1.39 \log_2 n$

Divide and Conquer

Mergesort, quicksort, BST traversal, strassen's algo, binarysearch

Quicksort: Best $\in \Theta(n \log n)$ Worst $\in \Theta(n^2)$ Avg $\in \Theta(n \log n)$

$C_B = 2C(\frac{n}{2}) + n \to$ partitioning $C_W = C(n-1) + n$ (sorted arr)

BST traversal $\in \Theta(n)$, height find $\in \Theta(n)$

$A = A_1 A_2$ } A_1, B_1 n digit
$B = B_1 B_2$ } $A_1 A_2 B_1 B_2$ n/2 digit

$M(n) = 4M(n/2), \ M(1) = 1$
$M(n) = n^2$

$A \times B = A_1 \times B_1 \cdot 10^n + (A_1 \times B_2 + A_2 \times B_1) \cdot 10^{n/2} + A_2 \cdot B_2$

2nd: $(A_1 \times B_2 + A_2 \times B_1) = (A_1 + A_2) \times (B_1 + B_2) - A_1 \times B_1 - A_2 \times B_2$

$M(n) = 3M(n/2), \ M(1) = 1 \quad M(n) = n^{\log_2 3} \approx n^{1.585}$

Closest pair: $T(n) = 2T(n/2) + M(n) \in \Theta(n \log n)$

Transform and Conquer

$\lceil \log_2 n! \rceil \approx n \log_2 n$ comparisons worst case to sort list of size n by any algorithm.

Searching with presort: 1) Sort $\Theta(n \log n)$ 2) Binarys. $O(\log n)$ } $\Theta(n \log n)$

BST operations: Worst $\in \Theta(n)$ Avg $\in \Theta(\log n)$

Heaps: n elements → $h = \lfloor \log_2 n \rfloor$ root is the largest

Build heap worst case: $C(n) = \sum_{i=0}^{h-1} 2(h-i)2^i = 2(n - \log_2(n+1)) \in \Theta(n)$

(n-1) times root removal worst case: $C(n) = \sum_{i=1}^{n-1} 2 \log_2 i \in \Theta(n \log n)$

Heapsort: Worst and avg $\in \Theta(n \log n)$ no stability

Insertion to heap $\in O(\log n)$

Transform and Conquer

Computing a^n

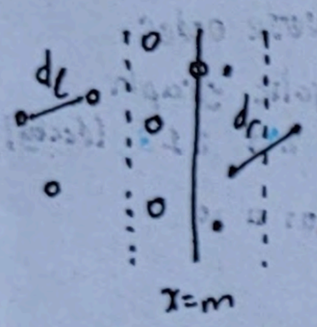
$a^{13} \rightarrow n=13 \rightarrow 1101_2$

1	1	0	1
SM	SM	S	SM
$1^2 \times a = a$	$a^2 \times 1 = a^3$	$(a^3)^2 = a^6$	$(a^6)^2 \times a = a^{13}$

$b \leq M(n) \leq 2b \quad b = \lfloor \log_2 n + 1 \rfloor$

Closest Pair by Divide & Conquer

1. Divide points into two subsets P_l, P_r by a vertical line $x=m$ so that half the points lie to the left or on the line and ~~the~~ half the points lie to the right or on the line.



2. Find recursively the closest pairs for the left and right subsets.
3. Set $d = \min\{d_l, d_r\}$ We can limit our attention to the points in the symmetric vertical strip S of width 2d as possible closest pair. (The points are stored and processed in increasing order of their y coordinates)
4. Scan the points in the vertical strip S from the lowest up. For every point $p(x,y)$ in the strip, inspect points in in the strip that may be closer to p than d. There can be no more than 5 such points following p on the strip list.

Efficiency: $T(n) = 2T(n/2) + M(n)$
$M(n) \in O(n) \quad T(n) \in \Theta(n \log n)$

Quickhull Algorithm

Convex hull: smallest convex set that includes given points
Assume points are sorted by x vals,
Identify extreme points P_1, P_2 (leftmost, rightmost)
Compute upper hull recursively:
Find point P_{max} that is farthest away from line $P_1 P_2$
Compute the upper hull of the points to the left of line $P_1 P_{max}$
Compute the upper hull of the points to the left of line $P_{max} P_2$
Compute lower hull in a similar manner
$C_W \in \Theta(n^2) \quad C_{Avg} \in \Theta(n)$

Common Recurrences

$$T(n) = T(n-1) + f(n)$$
$$= T(0) + \sum_{j=1}^{n} f(j)$$

$$T(n) = T(n/b) + f(n)$$
$n = b^k, \ b > 1$
$= T(b^{k-1}) + \dots$
$\vdots$
$= T(1) + \sum_{j=1}^{k} f(b^j)$

$$T(n) = aT(n/b) + f(n)$$
$n = b^k \quad a \geq 1 \quad b \geq 2$

$$T(n) = a^k \left(T(1) + \sum_{j=1}^{k} \frac{f(b^j)}{a^j}\right)$$

$$T(n) = n^{\log_b a} \left[T(1) + \sum_{j=1}^{\log_b a} \frac{f(b^j)}{a^j}\right]$$

Smoothness Rule

$f(n)$ is called smooth if it is defined on natural numbers and non negative, non decreasing
$f(2n) \in \Theta(f(n))$

Mergesort

```
Mergesort(A[0..n-1])
 if n>1
  copy A[0...⌊n/2⌋-1] to B[0..⌊n/2⌋-1]
  copy A[⌊n/2⌋....n-1] to B[0..⌈n/2⌉-1]
  Mergesort(B)
  Mergesort(C)
  Merge(B, C, A)

Merge(B[0..p-1], C[0..q-1], A[0..p+q-1])
 i,j,k = 0
 while i<p and j<q
  if B[i] ≤ C[j]
   A[k] ← B[i]; i++
  else
   A[k] ← C[j]; j++
  k++
 if i == p
  copy C[j...q-1] to A[k...p+q-1]
 else
  copy B[i...p-1] to A[k...p+q-1]
```

```
int i=n, x=0, count=0
while i>1 {
  x += 2
  i /= 3
} 2log_3 n
for j to x {
  for k to x
} x^2
(2 log_3 n)^2 ∈ Θ(log^2 n)
```

Big-O	Best	Avg.	Worst	Worst space
Quicksort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$	$O(\log n)$
Mergesort	"	"	$O(n \log n)$	$O(n)$
Heapsort	"	"	"	$O(1)$
Bubble sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Insertion sort	"	"	"	"
Selection sort	$\Omega(n^2)$	"	"	$O(1)$
Tree sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$	$O(n)$
Counting sort	$\Omega(n+k)$	$\Theta(n+k)$	$O(n+k)$	$O(k)$

Quicksort Partitioning

```
Partitioning (A[l...r])
 p = A[l]
 i = l, j = r+1
 repeat
  repeat i++ until A[i] ≥ p
  repeat j-- until A[j] ≤ p
  swap(A[i], A[j])
 until i ≥ j
 swap(A[i], A[j])
 swap(A[l], A[j])
 return j
```

Generating Subsets

Generating 2^n bit strings n-element set.

```
n==1 → make list L of two bits 0 and 1
else
 generate recursively L1 from n-1
 copy list L1 in reverse order for L2
 add 0 front of each string in L1
 add 1 " " " " " L2
 append L2 to L1 to get L
 return L.
```

Boyer-Moore

$d_1 = \max\{t_1(c) - k\}$

$d = \begin{cases} d_1 & k=0 \\ \max(d_1, d_2) & k>0 \end{cases}$

Dynamic Programming

- Coin Row

$F(0)=0 \quad F(1)=c_1$

$F(n) = \max\{c_n + F(n-2), F(n-1)\}$ for $n>1$

i) Set up recurrence
ii) Solve smaller instances
iii) Record in a table
iv) Extract sln. from table

- Coin Collecting

$F(i,j) = \max[F(i-1,j), F(i,j-1)] + c_{ij}$

$F(0,j)=0 \quad F(i,0)=0$

- Knapsack

$$V[i,j] = \begin{cases} \max\{V[i-1,j], v_i + V[i-1, j-w_i]\} & j - w_i \geq 0 \\ V[i-1,j] & j - w_i < 0 \end{cases}$$

i: items

Class P

Searching	$O(n)$
Element uniqueness	$O(n \log n)$
Graph connectivity	DFS, BFS
Graph acyclicity	DFS, BFS
Primality testing	$O(\log^7 n)$

Class NP

nondeterministic polynomial
(decision prob. can be verified in polynomial time)

Hamilton circuit existence
Partition prob.
Decision var. TSP, knapsack
graph coloring (except MST, sht. path)